

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Naufal Elyzar NIM. 231017210019

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
Juni 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Naufal Elyzar
NIM : 2310817210019

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom.
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	6
B. Output Program.....	21
C. Pembahasan	21
D. Tautan Git	23
SOAL 2.....	Error! Bookmark not defined.
A. Pembahasan	Error! Bookmark not defined.

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 1 XML	21
Gambar 2 Screenshot Hasil Jawaban Soal 1 Jetpack Compose	21

DAFTAR TABEL

Tabel 1 Source Code Jawaban Soal 1.....	10
Tabel 2 Source Code Jawaban Soal 1.....	11
Tabel 3. Source Code Jawaban Soal 1.....	14

Soal Praktikum:

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

MainActivity.kt

```
1 package com.example.modul4compose
2 ``
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.Box
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.material.icons.Icons
```

```
10 import androidx.compose.material.icons.filled.Favorite
11 import androidx.compose.material.icons.filled.Home
12 import androidx.compose.material.icons.filled.Settings
13 import androidx.compose.material3.*
14 import androidx.compose.runtime.Composable
15 import androidx.compose.ui.Modifier
16 import androidx.navigation.NavType
17 import androidx.navigation.compose.NavHost
18 import androidx.navigation.compose.composable
19 import androidx.navigation.compose.rememberNavController
20 import androidx.navigation.navArgument
21 import com.example.modul4compose.navigation.NavArguments
22 import com.example.modul4compose.navigation.Screen
23 import
  com.example.modul4compose.ui.screen.MoviesListScreen
24 import
  com.example.modul4compose.ui.screen.MovieDetailScreen
25 import
  com.example.modul4compose.ui.screen.WishlistScreen
26 import
  com.example.modul4compose.ui.theme.Modul4ComposeTheme
27 ``
28 class MainActivity : ComponentActivity() {
29     override fun onCreate(savedInstanceState: Bundle?) {
30         super.onCreate(savedInstanceState)
31         setContent {
32             Modul4ComposeTheme {
```

```
33 Surface(
34     modifier = Modifier.fillMaxSize(),
35     color = MaterialTheme.colorScheme.background
36 ) {
37     val navController = rememberNavController()
38     ``
39     NavHost(
40         navController = navController,
41         startDestination = Screen.GamesList.route
42     ) {
43         composable(Screen.GamesList.route) {
44             MainScaffold(navController = navController, currentRoute = Screen.GamesList.route) {
45                 MoviesListScreen(modifier = Modifier, navController = navController)
46             }
47         }
48         ``
49         composable(
50             route = Screen.MovieDetail.route,
51             arguments = listOf(
52                 navArgument("movieId") {
53                     type = NavType.IntType
54                 }
55             )
56         ) { backStackEntry ->
```



```
57 val movieId = backStackEntry.arguments?.getInt("movieId")
   ?: 0
58 MovieDetailScreen(
59     movieId = movieId,
60     navController = navController
61 )
62 }
63 ``
64 composable(Screen.Wishlist.route) {
65     MainScaffold(navController = navController, currentRoute
   = Screen.Wishlist.route) {
66         WishlistScreen()
67     }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 ``
76 @Composable
77 fun                                MainScaffold(navController:
   androidx.navigation.NavController, currentRoute: String,
   content: @Composable () -> Unit) {
78     Scaffold(
79     bottomBar = {
```

```

80  NavigationBar {
81  NavigationBarItem(
82  selected = currentRoute == Screen.GamesList.route,
83  onClick = { navController.navigate(Screen.GamesList.route) },
84  label = { Text("Movies") },
85  icon = { Icon(Icons.Default.Home, contentDescription = null) }
86  )
87  NavigationBarItem(
88  selected = currentRoute == Screen.Wishlist.route,
89  onClick = { navController.navigate(Screen.Wishlist.route) },
90  label = { Text("Wishlist") },
91  icon = { Icon(Icons.Default.Favorite, contentDescription = null) }
92  )
93  }
94  }
95  ) { innerPadding ->
96  Box(modifier = Modifier.padding(innerPadding)) {
97  content()
98  }
99  }
100 }

```

Tabel 1 Source Code Jawaban Soal 1

MovieDetailScreen.kt

```
1 package com.example.modul4compose.ui.screen
2 ``
3 import androidx.compose.foundation.layout.*
4 import androidx.compose.foundation.rememberScrollState
5 import androidx.compose.foundation.verticalScroll
6 import androidx.compose.material.icons.Icons
7 import androidx.compose.material.icons.filled.ArrowBack
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.layout.ContentScale
13 import androidx.compose.ui.platform.LocalContext
14 import androidx.compose.ui.unit.dp
15 import androidx.lifecycle.viewmodel.compose.viewModel
16 import androidx.navigation.NavController
17 import androidx.room.Room
18 import coil.compose.AsyncImage
19 import com.example.modul4compose.data.room.AppDatabase
20 import com.example.modul4compose.data.room.WishlistMovie
21 import
22 com.example.modul4compose.viewmodel.MoviesViewModel
23 import kotlinx.coroutines.launch
24 ``
25 @OptIn(ExperimentalMaterial3Api::class)
26 @Composable
27 fun MovieDetailScreen(
28     movieId: Int,
29     navController: NavController,
30     modifier: Modifier = Modifier,
31     viewModel: MoviesViewModel = viewModel()
32 ) {
33     val movie by viewModel.movieDetail.collectAsState()
34     val context = LocalContext.current
35     val db = remember {
36         Room.databaseBuilder(
37             context.applicationContext,
38             AppDatabase::class.java, "app_db"
39         ).fallbackToDestructiveMigration().build()
```



```
76 modifier = Modifier
77   .fillMaxWidth()
78   .height(350.dp),
79 contentScale = ContentScale.Crop
80 )
81 Spacer(modifier = Modifier.height(16.dp))
82 Text(
83   text = movie!!.title,
84   style = MaterialTheme.typography.headlineMedium
85 )
86 Spacer(modifier = Modifier.height(8.dp))
87 Text(
88   text = movie!!.overview,
89   style = MaterialTheme.typography.bodyLarge
90 )
91 Spacer(modifier = Modifier.height(16.dp))
92 Button(
93   onClick = {
94     scope.launch {
95       db.wishlistDao().insert(
96         WishlistMovie(
97           id = movie!!.id,
98           title = movie!!.title,
99           poster_path = movie!!.poster_path
100 )
101 )
102 addToWishlist = true
103 }
104 },
105 enabled = !addToWishlist
106 ) {
107   Text(if (addToWishlist) "Added to Wishlist" else "Add
to Wishlist")
108 }
109 }
110 } else {
111   CircularProgressIndicator()
112 }
113 }
```

114	}
115	}

Tabel 2 Source Code Jawaban Soal 1

MoviesListScreen.kt

1	package com.example.modul4compose.ui.screen
2	``
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.lazy.LazyColumn
5	import androidx.compose.foundation.lazy.items
6	import
7	androidx.compose.foundation.shape.RoundedCornerShape
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.runtime.collectAsState
11	import androidx.compose.runtime.getValue
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.draw.clip
15	import androidx.compose.ui.layout.ContentScale
16	import androidx.compose.ui.text.style.TextOverflow
17	import androidx.compose.ui.unit.dp
18	import androidx.lifecycle.viewmodel.compose.viewModel
19	import coil.compose.AsyncImage
20	import com.example.modul4compose.data.Movie
21	import
22	com.example.modul4compose.viewmodel.MoviesViewModel
23	import androidx.navigation.NavController
24	import com.example.modul4compose.navigation.Screen
25	``
26	@Composable
27	fun MoviesListScreen(
28	modifier: Modifier = Modifier,
29	viewModel: MoviesViewModel = viewModel(),
30	navController: NavController
31	) {
32	val movies by viewModel.movies.collectAsState()
33	``

```

32 LazyColumn(
33     modifier = modifier.fillMaxSize(),
34     contentPadding = PaddingValues(16.dp),
35     verticalArrangement = Arrangement.spacedBy(16.dp)
36 ) {
37     items(movies) { movie ->
38         MovieCard(movie = movie, navController = navController)
39     }
40 }
41 ``
42 viewModel.fetchMovies()
43 }
44 ``
45 @Composable
46 fun MovieCard(movie: Movie, navController:
    NavController) {
47     Card(
48         modifier = Modifier.fillMaxWidth(),
49         shape = RoundedCornerShape(16.dp)
50     ) {
51         Row(
52             modifier = Modifier
53                 .padding(16.dp)
54                 .fillMaxWidth(),
55             verticalAlignment = Alignment.CenterVertically
56         ) {
57             AsyncImage(
58                 model =
59                 "https://image.tmdb.org/t/p/w500${movie.poster_path}",
60                 contentDescription = movie.title,
61                 modifier = Modifier
62                     .size(width = 120.dp, height = 120.dp)
63                     .clip(RoundedCornerShape(12.dp)),
64                 contentScale = ContentScale.Crop
65             )
66             ``
67             Spacer(modifier = Modifier.width(16.dp))
68             ``
69             Column(modifier = Modifier.weight(1f)) {

```

```

69 Text(
70 text = movie.title,
71 style = MaterialTheme.typography.titleLarge
72 )
73 Text(
74 text = movie.overview,
75 style = MaterialTheme.typography.bodyMedium,
76 maxLines = 2,
77 overflow = TextOverflow.Ellipsis
78 )
79 Spacer(modifier = Modifier.height(8.dp))
80 Button(
81 onClick = {
82 navController.navigate(Screen.MovieDetail.createRoute(movie.id))
83 },
84 modifier = Modifier.align(Alignment.End)
85 ) {
86 Text(text = "Detail")
87 }
88 }
89 }
90 }
91 }

```

Tabel 3. Source Code Jawaban Soal 1

PersistenceScreen.kt

```

1 package com.example.modul4compose.ui.screen
2 ``
3 import androidx.compose.foundation.layout.*
4 import androidx.compose.foundation.lazy.LazyColumn
5 import androidx.compose.foundation.lazy.items
6 import androidx.compose.material.icons.Icons
7 import androidx.compose.material.icons.filled.Delete
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.platform.LocalContext

```



```
13 import androidx.compose.ui.unit.dp
14 import androidx.lifecycle.viewmodel.compose.viewModel
15 import androidx.room.Room
16 import coil.compose.AsyncImage
17 import com.example.modul4compose.data.room.AppDatabase
18 import com.example.modul4compose.data.room.WishlistMovie
19 import kotlinx.coroutines.launch
20 ``
21 @Composable
22 fun WishlistScreen() {
23     val context = LocalContext.current
24     val db = remember {
25         Room.databaseBuilder(
26             context.applicationContext,
27             AppDatabase::class.java, "app_db"
28         ).fallbackToDestructiveMigration().build()
29     }
30     var wishlist by remember {
31         mutableStateOf(listOf<WishlistMovie>()) }
32     val scope = rememberCoroutineScope()
33     ``
34     LaunchedEffect(Unit) {
35         wishlist = db.wishlistDao().getAll()
36     }
37     ``
38     Column(
39         modifier = Modifier
40             .fillMaxSize()
41             .padding(16.dp),
42         horizontalAlignment = Alignment.CenterHorizontally
43     ) {
44         Text("Wishlist Film", style =
45             MaterialTheme.typography.titleLarge)
46         Spacer(Modifier.height(16.dp))
47         LazyColumn(
48             modifier = Modifier.fillMaxWidth(),
49             verticalArrangement = Arrangement.spacedBy(12.dp)
50         ) {
51             items(wishlist, key = { it.id }) { movie ->
```

```

50 Card(
51     modifier = Modifier.fillMaxWidth(),
52     colors = CardDefaults.cardColors(containerColor =
53         MaterialTheme.colorScheme.surfaceVariant)
54 ) {
55     Row(
56         verticalAlignment = Alignment.CenterVertically,
57         modifier = Modifier.padding(12.dp)
58     ) {
59         AsyncImage(
60             model =
61             "https://image.tmdb.org/t/p/w500${movie.poster_path}",
62             contentDescription = movie.title,
63             modifier = Modifier.size(60.dp)
64         )
65         Spacer(Modifier.width(12.dp))
66         Text(movie.title, modifier = Modifier.weight(1f))
67         IconButton(onClick = {
68             scope.launch {
69                 db.wishlistDao().delete(movie)
70                 wishlist = db.wishlistDao().getAll()
71             }
72         }) {
73             Icon(Icons.Default.Delete, contentDescription =
74                 "Remove")
75         }
76     }
77 }
78 }

```

Tabel 4 Tabel Source Code Soal 1

MoviesViewModel.kt

```

1 package com.example.modul4compose.viewmodel
2 ``
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope

```

```
5 import kotlinx.coroutines.flow.MutableStateFlow
6 import kotlinx.coroutines.flow.StateFlow
7 import kotlinx.coroutines.launch
8 import com.example.modul4compose.data.Movie
9 import com.example.modul4compose.data.RetrofitInstance
10 import retrofit2.Response
11 import timber.log.Timber
12 ``
13 class MoviesViewModel : ViewModel() {
14     private val _movies =
15         MutableStateFlow<List<Movie>>(emptyList())
16     val movies: StateFlow<List<Movie>> = _movies
17     ``
18     private val _movieDetail =
19         MutableStateFlow<Movie?>(null)
20     val movieDetail: StateFlow<Movie?> = _movieDetail
21     ``
22     fun fetchMovies() {
23         viewModelScope.launch {
24             try {
25                 val response =
26                     RetrofitInstance.api.getPopularMovies("a9bba3f6f1cd96583
27                     db3855dd165da1")
28                 if (response.isSuccessful) {
29                     _movies.value = response.body()?.results ?: emptyList()
30                 } else {
31                     Timber.e("Error: ${response.message()}")
32                 }
33             } catch (e: Exception) {
34                 Timber.e("Error: ${e.message}")
35             }
36         }
37     }
38     ``
39     fun fetchMovieDetail(movieId: Int) {
40         viewModelScope.launch {
41             try {
42                 val response =
43                     RetrofitInstance.api.getMovieDetail(movieId,
44                     "a9bba3f6f1cd96583db3855dd165da1")
45             } catch (e: Exception) {
46                 Timber.e("Error: ${e.message}")
47             }
48         }
49     }
50 }
```

```

39  if (response.isSuccessful) {
40    _movieDetail.value = response.body()
41  } else {
42    Timber.e("Error: ${response.message()}")
43  }
44  } catch (e: Exception) {
45    Timber.e("Error: ${e.message}")
46  }
47  }
48  }
49  }

```

Tabel 5 Tabel Source Code Soal 1

NavGraph.kt

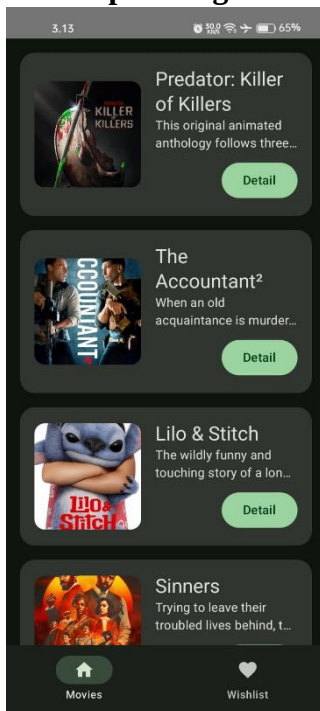
```

1  package com.example.modul4compose.navigation
2  ``
3  sealed class Screen(val route: String) {
4    object GamesList : Screen("games_list")
5    object GameDetail : Screen("game_detail/{gameId}") {
6      fun createRoute(gameId: Int) = "game_detail/$gameId"
7    }
8    object MovieDetail : Screen("movie_detail/{movieId}") {
9      fun createRoute(movieId: Int) =
10       "movie_detail/$movieId"
11    }
12    object Wishlist : Screen("wishlist")
13  }
14  ``
15  object NavArguments {
16    const val GAME_ID = "gameId"
17  }

```

Tabel 6 Tabel Source Code Soal 1

B. Output Program



Gambar 1 Screenshot Hasil Jawaban Soal 1



Gambar 2 Screenshot Hasil Jawaban Soal 1

C. Pembahasan MainActivity.kt:

Pada line 1, dideklarasikan package bernama `com.example.modul4compose` yang merupakan identitas unik dari aplikasi ini.

Pada line 3, diimpor `Bundle` yang digunakan untuk menyimpan dan mengelola data antar aktivitas.

Pada line 4, diimpor `ComponentActivity` yang digunakan untuk mendukung aktivitas berbasis Jetpack Compose.

Pada line 5, diimpor `setContent` untuk mendefinisikan tampilan UI menggunakan Jetpack Compose.

Pada line 6, diimpor `Box`, elemen layout untuk menampung dan menyusun UI secara fleksibel.

Pada line 7, diimpor `fillMaxSize` yang digunakan untuk mengisi seluruh ukuran layar.

Pada line 8, diimpor `padding` untuk memberikan jarak atau ruang antar elemen.

Pada line 9, diimpor ikon dari Jetpack Compose yaitu `Icons`.

Pada line 10, diimpor ikon `Favorite`, ikon hati yang menandakan favorit atau wishlist.

Pada line 11, diimpor ikon `Home`, ikon rumah yang biasa digunakan untuk menu utama.

Pada line 12, diimpor ikon `Settings`, meskipun dalam kode tidak digunakan.

Pada line 13, diimpor `material3.*` untuk menggunakan komponen UI berbasis Material Design 3 seperti `Surface`, `Text`, `NavigationBar`, dll.

Pada line 14, diimpor `@Composable` yang digunakan untuk mendeklarasikan fungsi composable di Jetpack Compose.

Pada line 15, diimpor `Modifier`, digunakan untuk mengatur tampilan seperti ukuran dan padding.

Pada line 16, diimpor `NavType` untuk mendefinisikan tipe argumen dalam navigasi.

Pada line 17, diimpor `NavHost` sebagai container utama untuk semua navigasi antar screen.

Pada line 18, diimpor `composable` untuk membuat rute navigasi menuju screen tertentu.

Pada line 19, diimpor `rememberNavController`, digunakan untuk menyimpan dan mengontrol status navigasi.

Pada line 20, diimpor `navArgument` untuk menetapkan argumen navigasi, seperti ID film.

Pada line 21, diimpor `NavArguments`, walaupun tidak digunakan dalam kode ini.

Pada line 22, diimpor `Screen`, class yang mendefinisikan route navigasi.

Pada line 23, diimpor `MoviesListScreen`, yaitu screen utama yang menampilkan daftar film.

Pada line 24, diimpor `MovieDetailScreen`, yaitu screen untuk menampilkan detail film berdasarkan ID.

Pada line 25, diimpor `WishlistScreen`, yaitu screen yang menampilkan daftar film favorit.

Pada line 26, diimpor `Modul4ComposeTheme`, yaitu tema khusus aplikasi ini.

Pada line 28, dideklarasikan class `MainActivity` yang merupakan turunan dari `ComponentActivity`, sebagai aktivitas utama.

Pada line 29, fungsi `onCreate` dioverride, dipanggil saat activity pertama kali dibuat.

Pada line 30, dipanggil `super.onCreate(savedInstanceState)` untuk menjalankan inisialisasi dari superclass.

Pada line 31, digunakan `setContent` untuk menentukan isi tampilan menggunakan Jetpack Compose.

Pada line 32, digunakan tema `Modul4ComposeTheme` untuk menetapkan tema aplikasi.

Pada line 33, digunakan `Surface` sebagai container utama dengan warna background dari tema.

Pada line 34, `Modifier.fillMaxSize()` digunakan agar tampilan mengisi seluruh layar.

Pada line 35, warna background ditentukan berdasarkan skema warna dari `MaterialTheme`.

Pada line 37, dibuat navController untuk mengatur navigasi antar screen.
Pada line 39, digunakan NavHost sebagai container navigasi dengan navController.
Pada line 40, parameter navController diteruskan ke NavHost.
Pada line 41, startDestination diset ke Screen.GameList.route sebagai tampilan awal.
Pada line 43, dibuat rute navigasi untuk GameList menggunakan fungsi composable.
Pada line 44, dipanggil MainScaffold, wrapper layout dengan navigasi bawah dan konten utama.
Pada line 45, MovieListScreen ditampilkan dengan Modifier dan navController.
Pada line 49, dibuat rute baru untuk MovieDetail dengan argumen movieId.
Pada line 50, route diisi dari Screen.MovieDetail.route.
Pada line 51, arguments berisi daftar navArgument.
Pada line 52, dideklarasikan argumen movieId dengan tipe NavType.IntType.
Pada line 56, definisi isi rute MovieDetail, menerima backStackEntry.
Pada line 57, movieId diambil dari argumen navigasi, default ke 0 jika null.
Pada line 58, MovieDetailScreen ditampilkan dengan movieId dan navController.
Pada line 63, dibuat rute ketiga untuk Wishlist.
Pada line 64, digunakan MainScaffold dengan currentRoute diset ke Wishlist.
Pada line 65, di dalamnya dipanggil WishlistScreen().
Pada line 76, dideklarasikan fungsi composable MainScaffold.
Pada line 77, fungsi menerima navController, currentRoute, dan content sebagai parameter.
Pada line 78, digunakan Scaffold sebagai layout utama dengan bottomBar.
Pada line 79, bottomBar didefinisikan menggunakan NavigationBar.
Pada line 80, NavigationBar adalah baris navigasi bawah layar.
Pada line 81, ditambahkan item navigasi pertama.
Pada line 82, item pertama dipilih jika rute saat ini adalah GameList.
Pada line 83, jika diklik, akan menavigasi ke GameList.
Pada line 84, label ditampilkan berupa teks "Movies".
Pada line 85, ikon ditampilkan berupa Icons.Default.Home.
Pada line 87, ditambahkan item navigasi kedua.
Pada line 88, item kedua aktif jika currentRoute adalah Wishlist.
Pada line 89, jika diklik, akan menavigasi ke rute Wishlist.
Pada line 90, label ditampilkan berupa teks "Wishlist".
Pada line 91, ikon ditampilkan berupa Icons.Default.Favorite.
Pada line 95, isi dari Scaffold akan diposisikan sesuai padding innerPadding.
Pada line 96, digunakan Box sebagai container isi utama.
Pada line 97, content() akan dipanggil di dalam Box, sesuai parameter fungsi.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NaufalElyzar11/PraktikumMobile/tree/main/Modul%205>